Infrastructure & Deployment Guide

FieldPilot on AWS

Production Deployment Strategy

Prepared for: FieldPilot / FieldRino Team

Budget Target: $40 / month

Date: March 2026

Status: Recommended Architecture

1. Honest Assessment

⚠️ Budget vs. Scale Trade-off

$40/month vs. "thousands of users + zero downtime" — there is an important tension here. True high-availability (multi-AZ, auto-scaling, ALB) on AWS costs **$80–120/month minimum**. At $40/month you can serve thousands of *registered* users (not thousands of simultaneous concurrent users) reliably — but with a single point of failure on the application server.

Requirement	$40/mo Setup	True HA Setup ($80–120/mo)
Thousands of registered users	✓ Yes	✓ Yes
Hundreds of daily active users	✓ Yes	✓ Yes
App server high-availability	✗ No (single EC2)	✓ Yes (ALB + 2 EC2s)
Database failover (Multi-AZ)	✗ No	✓ Yes
Zero-downtime deploys	Partial	✓ Yes
Auto-scaling under load	✗ No	✓ Yes

2. Recommended Architecture (~$40/month)

```
┌──────────────────────────────────────┐ │ INTERNET │
```

```
└──────────┬──────────────────────────┘ │ ┌──────────▼──────────┐ │ Route 53 (DNS)
```

```
│ │ *.domain.com → EC2 IP │ └──────────┬──────────────────────┘ │
```

```
┌──────────▼──────────┐ │ ACM (Free SSL/TLS) │ └──────────┬──────────────────────┘
```

```
│ ┌──────────────────────▼──────────────────────┐ │ EC2 t3a.small (2 vCPU / 2
```

```
GB RAM) │ │ │ │ Nginx ──▶ Gunicorn (Django) │ │ Celery Worker │ │ Celery Beat │
```

```
│ Supervisor (process manager) │
```

```
└──────┬──────────────────────┬──────────────────────┘ │ │ ┌──────────▼──────────┐
```

```
┌──────▼──────────────┐ │ RDS PostgreSQL │ │ ElastiCache Redis │ │
```

```
db.t3.micro │ │ cache.t3.micro │ │ (django-tenants) │ │ (Celery + caching) │
```

```
└──────────────────────┘ └──────────────────────┘ │
```

```
┌──────────────▼──────────────────────────────┐ │ S3 ──▶ CloudFront CDN │ │
```

```
(Media files, Static assets, PDF reports) │
```

```
└──────────────────────────────────────────────┘
```

3. Service-by-Service Cost Breakdown

AWS Service	Tier / Config	On-Demand/mo	1-Yr Reserved/mo	Purpose
EC2 t3a.small	2 vCPU, 2 GB RAM	~$13.50	~$8.50	Django + Celery + Nginx (all-in-one)
RDS PostgreSQL db.t3.micro	Single-AZ, 20 GB gp3	~$12.50	~$8.00	Primary DB — multi-tenant schemas
ElastiCache cache.t3.micro	Redis, single-node	~$12.50	~$8.50	Celery task queue + app caching

AWS Service	Tier / Config	On-Demand/mo	1-Yr Reserved/mo	Purpose
S3 Standard	Pay-per-use	~$1–2	~$1–2	Media files, static assets, report exports
CloudFront	Pay-per-use CDN	~$0.50–1	~$0.50–1	CDN for static/media — reduces EC2 load
Route 53	Hosted zone	~$0.50	~$0.50	DNS + subdomain routing for multi-tenancy
ACM (SSL)	Wildcard cert	Free	Free	SSL/TLS certificates
SES	$0.10/1000 emails	~$0.10–1	~$0.10–1	OTPs, notifications, report delivery
TOTAL (On-Demand)		**~$40–42/mo**	**~$27–29/mo**	1-yr reserved saves ~$13/month

💡 Pro Tip: Use Reserved Instances

Purchase 1-year reserved (no upfront) for EC2, RDS, and ElastiCache. This drops your total bill to **~$27–29/month** — saving ~$13/month and giving you headroom for S3, CloudFront, SES, and Route 53 within $40.

4. EC2 Server Setup

All application processes run on a single EC2 t3a.small instance managed by Supervisor:

```
EC2 t3a.small (2 vCPU, 2 GB RAM) ├── Nginx → Reverse proxy, SSL termination,
```

```
static file serving ├── Gunicorn → Django WSGI server (4 workers) ├── Celery
```

```
Worker → Background tasks: report generation, emails, file ops ├── Celery Beat →
```

```
Scheduled tasks: cron jobs, report scheduling └── Supervisor → Process manager —
```

```
auto-restarts crashed processes
```

Gunicorn Configuration

```
# gunicorn.conf.py workers = 4 # 2 × vCPUs (standard rule) worker_class = "sync" timeout = 120 keepalive = 5 max_requests = 1000 # Restart workers periodically (prevent memory leaks) max_requests_jitter = 100 bind = "0.0.0.0:8000" accesslog = "/var/log/gunicorn/access.log" errorlog = "/var/log/gunicorn/error.log"
```

Storage

- Use **EBS gp3 volume** (30 GB) — faster than gp2 at same cost (~$2.40/month)
- Enable **EC2 Auto Recovery** (free) — automatically restarts the instance on underlying hardware failure
- Assign an **Elastic IP** (free while attached) — static IP that survives reboots

5. RDS PostgreSQL Configuration

Your app uses **django-tenants** with schema-per-tenant isolation. The database is the most critical component.

```
# Key RDS settings for db.t3.micro max_connections = 100 # Hard limit on t3.micro shared_buffers = 128MB effective_cache_size = 384MB maintenance_work_mem = 64MB # Django DATABASE settings CONN_MAX_AGE = 60 # Persistent connections (reduce overhead) OPTIONS: connect_timeout = 10
```

RDS Setting	Recommended Value
Automated Backups	7-day retention (free within storage limit)
Backup Window	Low-traffic hours (e.g., 02:00–03:00 UTC)
Maintenance Window	Off-peak hours
Performance Insights	Enable (free tier: 7 days retention)
Storage Auto Scaling	Enable — max threshold 100 GB
Deletion Protection	Enable
Snapshot before deploy	Always — before every deployment

6. ElastiCache Redis Configuration

Redis serves two purposes in this application: **Celery task queue** and **Django caching** (report caching, session data).

```
# Django settings for Redis CELERY_BROKER_URL = "redis://your-elasticache-endpoint:6379/0" CELERY_RESULT_BACKEND = "redis://your-elasticache-endpoint:6379/0" CACHES = { "default": { "BACKEND": "django_redis.cache.RedisCache", "LOCATION": "redis://your-elasticache-endpoint:6379/1", "OPTIONS": { "CLIENT_CLASS": "django_redis.client.DefaultClient", "SOCKET_CONNECT_TIMEOUT": 5, "SOCKET_TIMEOUT": 5, } } }
```

7. S3 + CloudFront Setup

S3 stores all media files, static assets, and generated reports (PDF/Excel). CloudFront serves them globally at low latency, reducing load on your EC2.

```
# Django settings — django-storages with S3 DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage' STATICFILES_STORAGE = 'storages.backends.s3boto3.S3StaticStorage' AWS_STORAGE_BUCKET_NAME = 'fieldrino-media-prod' AWS_S3_REGION_NAME = 'ap-south-1' # Mumbai — closest for India AWS_S3_CUSTOM_DOMAIN = 'cdn.yourdomain.com' # CloudFront domain AWS_DEFAULT_ACL = 'private' AWS_S3_FILE_OVERWRITE = False AWS_S3_OBJECT_PARAMETERS = {'CacheControl': 'max-age=86400'}
```

S3 Bucket	Purpose	ACL
fieldrino-media-prod	User uploads, report files, equipment images	Private
fieldrino-static-prod	Django static files (JS, CSS, icons)	Public via CloudFront

8. Multi-Tenant Subdomain Routing

Your app uses subdomain-based tenant detection (e.g., `tenant1.yourdomain.com`). Route 53 must route all subdomains to your EC2 instance.

DNS Record	Type	Value
yourdomain.com	A	EC2 Elastic IP
*.yourdomain.com	A	EC2 Elastic IP (wildcard — covers all tenants)
api.yourdomain.com	A	EC2 Elastic IP
cdn.yourdomain.com	CNAME	CloudFront distribution domain

ℹ️ ACM Wildcard Certificate

Request a wildcard SSL certificate from AWS ACM: ***.yourdomain.com** This single free certificate covers all tenant subdomains automatically. Attach it to your Nginx config (via Certbot or import into ACM for CloudFront).

9. Security Configuration

EC2 Security Group Rules

Port	Protocol	Source	Purpose
80	TCP	0.0.0.0/0	HTTP (redirect to HTTPS)
443	TCP	0.0.0.0/0	HTTPS (Nginx)

Port	Protocol	Source	Purpose
22	TCP	Your IP only	SSH access

RDS Security Group

Port	Protocol	Source
5432	TCP	EC2 Security Group ID only

ElastiCache Security Group

Port	Protocol	Source
6379	TCP	EC2 Security Group ID only

IAM Role for EC2

Attach an IAM Role to EC2 with the following policies — **never hardcode AWS credentials**:

- **AmazonS3FullAccess** (or scoped to your bucket ARN)
- **AmazonSESFullAccess** (or SES send-only policy)
- **CloudWatchAgentServerPolicy** (for metrics and logs)

10. Monitoring & Alerts (Free / Low Cost)

Tool	Cost	What to Monitor
CloudWatch Metrics	Free tier	EC2 CPU, RDS connections, ElastiCache memory
CloudWatch Agent	Free	EC2 RAM usage, disk space (not default metrics)

Tool	Cost	What to Monitor
CloudWatch Alarms	10 free alarms	CPU > 80%, DB connections > 80, disk > 75%
CloudWatch Logs	5 GB free/mo	Django logs, Nginx access/error logs, Celery logs
RDS Performance Insights	Free (7-day)	Slow queries, wait events, connection counts
Sentry (free tier)	Free	Application errors and exceptions

Key CloudWatch Alarms to Set Up

- **EC2 CPU > 80%** for 5 minutes → SNS email alert
- **EC2 StatusCheckFailed** → triggers Auto Recovery
- **RDS FreeStorageSpace < 2 GB** → email alert
- **RDS DatabaseConnections > 80** → email alert
- **ElastiCache Evictions > 100** → email alert

11. Deployment Process (Zero Risk)

1. Take a **manual RDS snapshot** before every deployment (takes ~2 minutes, free)
2. SSH into EC2, pull latest code from git repository
3. Install dependencies: `pip install -r requirements.txt`
4. Run migrations: `python manage.py migrate --noinput`
5. Collect static files: `python manage.py collectstatic --noinput`
6. Reload Gunicorn gracefully: `sudo supervisorctl restart gunicorn` (existing connections finish before restart)
7. Reload Celery workers: `sudo supervisorctl restart celery`
8. Verify health endpoint responds: `curl https://yourdomain.com/api/health/`

ℹ️ Minimizing Downtime During Deploys

Use Gunicorn's **--graceful-timeout** (default 30s) — it lets active requests finish before restarting

workers. Total downtime is effectively 0–5 seconds. Schedule deployments during low-traffic hours for safety.

12. When to Upgrade (Scale Signals)

Signal / Trigger	Upgrade Action	Additional Cost
EC2 CPU consistently > 70%	Upgrade to t3a.medium (4 GB RAM)	+$13/mo
Memory pressure / OOM kills	Upgrade to t3a.medium	+$13/mo
DB connections maxing out (100)	Add PgBouncer on EC2 OR upgrade to db.t3.small	+$12/mo
Need true zero-downtime deploys	Add ALB + 2nd EC2 instance	+$30/mo
DB failover requirement (SLA)	Upgrade to db.t3.micro Multi-AZ	+$12/mo
Celery overloaded / task queue lag	Separate Celery to its own t3a.micro EC2	+$8/mo
Thousands of concurrent users	ECS Fargate + ALB + Multi-AZ RDS	~$80–120/mo total

13. True HA Architecture (~$90–110/month)

When your business grows and revenue justifies it, this is the proper production-grade HA setup:

```
┌──────────────────────────────────────────────┐ │ INTERNET │
```

```
└──────────────┬──────────────────────────────┘ │ ┌──────────────▼──────────────┐ │ CloudFront
```

```
+ WAF │ └──────────────┬──────────────────────────────┘ │ ┌──────────────▼──────────────┐ │
```

```
ALB (Application LB) │ │ (Multi-AZ, SSL termination) │
```

```
└──────┬──────────────────┬──────────┘ │ │ ┌──────────────▼─┐
```

```
┌──────▼──────────────┐ │ EC2 t3a.small │ │ EC2 t3a.small │ │ (AZ 1 — Primary)
```

```
│ │ (AZ 2 — Standby) │ └──────────────┬─┘ └──────┬──────────────┘ │ │
```

```
┌──────────────▼──────────────▼──────────────┐ │ RDS PostgreSQL db.t3.small
```

```
Multi-AZ │ │ (Primary AZ1) ──failover── (Standby AZ2) │
```

```
└──────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────┐ │ ElastiCache Redis
```

```
cache.t3.small │ └──────────────────────────────────────────┘
```

Service	Tier	Cost/mo
ALB	Application Load Balancer	~$16
EC2 × 2	t3a.small × 2 (reserved)	~$17
RDS Multi-AZ	db.t3.small (reserved)	~$28
ElastiCache	cache.t3.micro (reserved)	~$8.50
S3 + CloudFront + SES + Route 53	—	~$3–5
Total HA Setup		**~$72–75/mo**

14. Summary & Recommended Action Plan

✅ Start Here — $40/month Production Setup

EC2 t3a.small + RDS db.t3.micro + ElastiCache cache.t3.micro + S3 + CloudFront + Route 53 + ACM + SES. Buy 1-year reserved instances for EC2, RDS, and ElastiCache to bring the bill to ~$27–29/month.

Phase 1: Launch (Month 1) — ~$40/mo

- Set up VPC with public + private subnets across 2 AZs
- Launch EC2 t3a.small with Nginx + Gunicorn + Supervisor
- Create RDS PostgreSQL db.t3.micro (single-AZ, automated backups on)
- Create ElastiCache Redis cache.t3.micro
- Configure S3 buckets + CloudFront distribution
- Set up Route 53 wildcard DNS for multi-tenant subdomains
- Configure ACM wildcard SSL certificate
- Set up SES for transactional emails
- Configure CloudWatch alarms for CPU, memory, storage, connections

Phase 2: Optimize (Month 2–3) — ~$27/mo

- Purchase 1-year reserved instances for EC2, RDS, ElastiCache
- Add PgBouncer for PostgreSQL connection pooling
- Integrate Sentry for error tracking
- Set up automated deployment scripts (GitHub Actions → SSH deploy)

Phase 3: Scale (When Revenue Allows) — ~$75-110/mo

- Add ALB and second EC2 instance for high-availability
- Upgrade RDS to Multi-AZ for database failover
- Separate Celery to its own instance
- Consider ECS Fargate for auto-scaling

⚠️ Key Limitations to Communicate to Stakeholders

At $40/month, this architecture handles **hundreds of daily active users** and **thousands of total registered users** comfortably. However, it has a single app server and single-AZ database — meaning a hardware failure or AZ outage could cause 5–15 minutes of downtime. For guaranteed 99.9% uptime SLA, budget $75–110/month.